

Code Sample

Selected Replication Code

Rishav Roy

CODING SAMPLE: FULL COPY

These excerpts are selected from the replication code for the paper. The [full paper](#) and [repository](#) are available online. This PDF can be generated using `make samples` or `bash scripts/run_full_build.sh`.

Survey-weighted enrollment model

File: R/selection/estimate_selection_probit.R

```
selection_probit_variables <- function(selection_data) {
  controls <- c("AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR")
  exclusion_all_kids <- c("DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ")
  exclusion_district <- c(
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",
    "dmean_num_ENROLLMENT_COST"
  )
  intersect(c(controls, exclusion_all_kids, exclusion_district), names(selection_data))
}

## project the selection sample to the model's actual dependency surface
##
## Keeping unrelated Block-5 schooling attributes in the fitted survey design makes
## the serialized model change whenever those attributes change, even though they
## are absent from the probit formula. A narrow model frame lets {targets} skip the
## expensive AME branch when upstream changes do not affect estimation.
project_selection_model_data <- function(selection_data) {
  selection_data <- safe_df(selection_data)
  design <- c("enrolled", "FSU_SL_NO", "weight", "STATE", "STRATUM", "SUB_STRATUM_NO")
  keep <- unique(c(design, selection_probit_variables(selection_data)))
  selection_data[intersect(keep, names(selection_data))]
}

## estimate selection probit
##
estimate_selection_probit <- function(selection_data, cfg) {
```

```

if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
  return(list(status = "out_of_active_pipeline", reason = "No enrolled variable. "))
}
covars <- selection_probit_variables(selection_data)
if (!length(covars)) {
  return(list(status = "out_of_active_pipeline", reason = "No probit covariates. "))
}
f_probit <- stats::reformulate(covars, response = "enrolled")
if (identical(cfg$mode, "final") && requireNamespace("survey", quietly = TRUE)) {
  old_options <- options(survey.lonely.psu = "average")
  on.exit(options(old_options), add = TRUE)

  design <- build_survey_design_selection(selection_data)
  if (!is.null(design)) {
    out <- fit_selection_probit(design, f_probit)
    return(stabilize_selection_model_formula(out, f_probit))
  }
}
out <- stats::glm(f_probit, data = selection_data, family = stats::binomial(link = "probit"))
stabilize_selection_model_formula(out, f_probit)
}

#' store a durable selection-model formula
#'
#' Programmatically fitted models otherwise retain a call to the local symbol
#' `f_probit`. Packages which reconstruct model data from the saved call cannot
#' resolve that symbol after the model is serialized by targets. Embed the
#' formula object in the call and retain the existing audit attribute.
stabilize_selection_model_formula <- function(model, formula) {
  if (!inherits(formula, "formula")) {
    stop("Selection model formula must inherit from formula.", call. = FALSE)
  }
  if (!is.null(model$call)) model$call$formula <- formula
  attr(model, "selection_probit_formula") <- formula
  model
}

#' build survey design selection
#'
build_survey_design_selection <- function(selection_df) {
  psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(c("STATE", "STRATUM", "SUB_STRATUM_NO"), names(selection_df))
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) return(NULL)
  selection_df$survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),
    data = selection_df,

```

```

    nest = TRUE
  )
}

#' fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(f_probit, design = selection_design, family = stats::quasibinomial(link = "probit"))
}

```

Average marginal effects on the model estimation sample

File: R/selection/compute_average_marginal_effects.R

```

ame_model_weight <- function(model_data, model_weights = NULL) {
  weight_cols <- intersect(c("weight", "WEIGHT", "Multiplier", "multiplier", "(weights)"), names(model_data))
  if (length(weight_cols)) {
    return(suppressWarnings(as.numeric(model_data[[weight_cols[[1]]]])))
  }

  if (!is.null(model_weights) && length(model_weights) == nrow(model_data)) {
    return(suppressWarnings(as.numeric(model_weights)))
  }

  NULL
}

#' build marginaleffects newdata and weights
#'
#' @return List with `data`, `wts`, and `has_explicit_weights`.
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))
  model_weights <- tryCatch(stats::weights(model), error = function(e) NULL)
  weight <- ame_model_weight(model_data, model_weights)

  if (!is.null(weight) && length(weight) == nrow(model_data) && any(is.finite(weight) & weight != 0)) {
    model_data$.ame_weight <- weight
    return(list(data = model_data, wts = ".ame_weight", has_explicit_weights = TRUE))
  }

  list(data = model_data, wts = FALSE, has_explicit_weights = FALSE)
}

#' run marginaleffects while muffling only a redundant explicit-weight warning
#'
run_avg_slopes <- function(model, model_data, wts, numderiv = NULL) {
  args <- list(

```

```

    model = model,
    newdata = model_data,
    wts = wts,
    vcov = TRUE,
    type = "response"
  )
  if (!is.null(numderiv)) args$numderiv <- numderiv

  withCallingHandlers(
    do.call(marginaleffects::avg_slopes, args),
    warning = function(w) {
      msg <- conditionMessage(w)
      explicit_weight_warning <- grepl(
        "normally good practice to specify weights using the `wts` argument",
        msg,
        fixed = TRUE
      )
      if (explicit_weight_warning && !identical(wts, FALSE)) {
        invokeRestart("muffleWarning")
      }
    }
  )
}

#' compute ames autodiff
#'
compute_ames_autodiff <- function(model) {
  # Use the exact estimation sample retained by glm/svyglm. Passing the full
  # pre-model selection_data can have extra rows omitted during model fitting,
  # which makes marginaleffects recycle weights/covariates silently.
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(model, amed$data, amed$wts)
}

```

Montiel Olea–Pflueger effective-F calculation

File: R/iv/weak_identification.R

```

mop_effective_f <- function(
  model, analysis_data, tau = 0.10, size = 0.05) {
  unavailable <- function(reason) {
    list(
      statistic = NA_real_, critical_value = NA_real_, p.value = NA_real_,
      effective_df = NA_real_, tau = tau, size = size,
      status = "not_estimated", reason = reason
    )
  }

  if (!inherits(model, "ivreg")) {

```

```

    return(unavailable("Montiel Olea-Pflueger effective F requires an ivreg model."))
  }
  if (!requireNamespace("momentfit", quietly = TRUE)) {
    return(unavailable("Package 'momentfit' is not installed."))
  }
  if (!is.finite(tau) || tau <= 0 || !is.finite(size) || size <= 0 || size >= 1) {
    return(unavailable("Montiel Olea-Pflueger tau and size must be valid probabilities."))
  }

  terms <- parse_iv_formula_terms(model)
  if (is.null(terms)) {
    return(unavailable("Could not parse IV formula for Montiel Olea-Pflueger effective F."))
  }
  endogenous <- setdiff(terms$regressors, terms$instruments)
  if (length(endogenous) != 1L) {
    return(unavailable("Montiel Olea-Pflueger effective F requires exactly one endogenous regressor."))
  }

  formula <- stats::formula(model)
  regression_formula <- stats::as.formula(
    call("~", formula[[2L]], formula[[3L]][[2L]]),
    env = environment(formula)
  )
  instrument_formula <- stats::as.formula(
    call("~", formula[[3L]][[3L]]),
    env = environment(formula)
  )
  needed <- unique(c(all.vars(regression_formula), all.vars(instrument_formula)))
  data <- iv_analysis_frame(analysis_data, needed)
  missing <- setdiff(needed, names(data))
  if (length(missing)) {
    return(unavailable(paste(
      "Missing variables for Montiel Olea-Pflueger effective F:",
      paste(missing, collapse = ", ")
    )))
  }
  moment_data <- data[needed]
  if (any(!stats::complete.cases(moment_data))) {
    return(unavailable("Montiel Olea-Pflueger analysis data contain incomplete model rows."))
  }

  stored_cluster <- attr(model, "cluster_state", exact = TRUE)
  cluster <- if (
    length(stored_cluster) == stats::nobs(model) &&
    !anyNA(stored_cluster) && length(unique(stored_cluster)) >= 2L
  ) {
    as.vector(stored_cluster)
  } else {
    iv_model_cluster(model, data)
  }

```

```

covariance <- if (is.null(cluster)) "MDS" else "CL"
covariance_options <- if (is.null(cluster)) {
  list(type = "HCO")
} else {
  # momentfit's clustered moment covariance is based on sandwich::meatCL()
  # and supports HCO. Keep its package-native finite-cluster correction
  # rather than altering the existing HCl Wald-F diagnostic to force equality.
  list(
    cluster = data.frame(cluster = cluster),
    type = "HCO", cadjust = TRUE, multi0 = FALSE
  )
}

result <- tryCatch({
  moment_model <- momentfit::momentModel(
    regression_formula,
    instrument_formula,
    data = moment_data,
    vcov = covariance,
    vcovOptions = covariance_options
  )
  momentfit::MOPtest(
    moment_model,
    tau = tau,
    size = size,
    estMethod = "TSLS",
    simplified = TRUE,
    print = FALSE
  )
}, error = function(e) e)
if (inherits(result, "error")) return(unavailable(conditionMessage(result)))

values <- suppressWarnings(as.numeric(result[c("Feff", "critValue", "pvalue", "Keff")]))
names(values) <- c("statistic", "critical_value", "p.value", "effective_df")
if (length(values) != 4L || any(!is.finite(values))) {
  return(unavailable("momentfit::MOPtest() did not return finite effective-F diagnostics."))
}

c(
  as.list(values),
  list(tau = tau, size = size, status = "estimated", reason = NA_character_)
)
}

```

Anderson–Rubin inference and bounded exclusion sensitivity

File: R/iv/weak_identification.R

```

anderson_rubin_test <- function(
  data, outcome, treatment, excluded, included = character(),
  controls = character(), fixed_effect = "none", cluster, beta0 = 0
) {
  transformed <- ".ar_outcome"
  data[[transformed]] <- num(data[[outcome]]) - beta0 * num(data[[treatment]])
  rhs <- unique(c(excluded, included, controls, iv_fixed_effect_terms(fixed_effect)))
  fit <- stats::lm(stats::reformulate(rhs, response = transformed), data = data)
  test <- clustered_joint_wald_test(fit, excluded, cluster)
  c(statistic = test[["statistic"]], p.value = test[["p.value"]])
}

# Weak-IV-robust exclusion sensitivity for one scalar excluded instrument.
# For a candidate structural effect beta, regress Y - beta D on Z and the same
# nuisance terms used by the registered IV specification. The coefficient on Z
# is the direct effect gamma that would reconcile that beta with the data.
# Subtracting any fixed gamma * Z changes that coefficient but not the residuals,
# so the clustered standard error is invariant to gamma. This makes the union of
# Anderson--Rubin confidence sets over gamma in [lower, upper] available without
# a numerical gamma grid.
bounded_exclusion_ar_profile <- function(
  data, outcome, treatment, excluded, included = character(),
  controls = character(), fixed_effect = "none", cluster, beta_values) {
  excluded <- plain_chr(excluded)
  if (length(excluded) != 1L || is.na(excluded[[1L]]) || !nzchar(excluded[[1L]])) {
    stop("Bounded exclusion sensitivity requires exactly one excluded instrument.", call. = FALSE)
  }
  beta_values <- num(beta_values)
  if (!length(beta_values) || any(!is.finite(beta_values))) {
    stop("Bounded exclusion sensitivity requires finite beta values.", call. = FALSE)
  }
  instrument <- excluded[[1L]]
  rhs <- unique(c(instrument, included, controls, iv_fixed_effect_terms(fixed_effect)))

  safe_bind_rows(lapply(beta_values, function(beta0) {
    transformed <- ".bounded_exclusion_ar_outcome"
    x <- as.data.frame(data)
    x[[transformed]] <- num(x[[outcome]]) - beta0 * num(x[[treatment]])
    fit <- stats::lm(stats::reformulate(rhs, response = transformed), data = x)
    inference <- iv_clustered_inference(fit, cluster)
    term <- model_term_inference(fit, instrument, inference$vcov)
    data.frame(
      beta = beta0,
      direct_effect_estimate = term[["estimate"]],
      direct_effect_std.error = term[["std.error"]],
      residual_df = stats::df.residual(fit),
      status = if (
        identical(inference$status, "estimated") &&
        all(is.finite(term[c("estimate", "std.error")])) &&
        term[["std.error"]] > 0

```

```

    ) "estimated" else "inference_unavailable",
    reason = if (identical(inference$status, "unavailable")) inference$reason else NA_character_,
    stringsAsFactors = FALSE
  )
}))
}

```

Linked state-sector price series

File: R/prices/build_temporal_price_series.R

```

build_temporal_price_series <- function(
  price_sources,
  switch_date = as.Date("2013-01-01"),
  overlap_start = as.Date("2013-01-01"),
  overlap_end = as.Date("2014-12-01"),
  minimum_link_months = 6L,
  pre_switch_start = NULL,
  pre_switch_end = NULL) {
  switch_date <- price_boundary(switch_date)
  old <- select_pre_2013_price_series(price_sources)
  new <- select_post_2013_price_series(price_sources)
  links <- summarise_price_links(old, new, overlap_start, overlap_end)

  minimum_link_months <- as.integer(minimum_link_months)
  if (length(minimum_link_months) != 1L || is.na(minimum_link_months) || minimum_link_months < 1L)
    stop("minimum_link_months must be one positive integer.", call. = FALSE)
}

old_groups <- unique(old[old$period < switch_date, c("state_code", "sector"), drop = FALSE])
linked_groups <- links[links$link_months >= minimum_link_months, c("state_code", "sector"), drop = FALSE]
linked_key <- paste(linked_groups$state_code, linked_groups$sector, sep = "\r")
missing_key <- !paste(old_groups$state_code, old_groups$sector, sep = "\r") %in% linked_key
if (any(missing_key)) {
  bad <- old_groups[missing_key, , drop = FALSE]
  stop(
    "Pre-2013 price series lack a sufficient direct link to CPI-R/U: ",
    paste(paste(bad$state_code, bad$sector, sep = "/"), collapse = ", "),
    call. = FALSE
  )
}

old <- merge(old, links, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
old <- old[old$period < switch_date, , drop = FALSE]
if (!is.null(pre_switch_start) || !is.null(pre_switch_end)) {
  if (is.null(pre_switch_start) || is.null(pre_switch_end)) {
    stop("Both pre_switch_start and pre_switch_end are required when trimming the old price series")
  }
  pre_switch_start <- price_boundary(pre_switch_start)

```

```

pre_switch_end <- price_boundary(pre_switch_end)
if (pre_switch_end < pre_switch_start || pre_switch_end >= switch_date) {
  stop("The pre-switch production window must end before the switch date.", call. = FALSE)
}
old <- old[old$period >= pre_switch_start & old$period <= pre_switch_end, , drop = FALSE]
}
old$index_unlinked <- old$index
old$index <- num(old$index) * num(old$link_factor)

new <- new[new$period >= switch_date, , drop = FALSE]
new$link_factor <- 1
new$link_months <- NA_integer_
new$link_ratio_mad <- NA_real_
new$link_ratio_min <- NA_real_
new$link_ratio_max <- NA_real_
new$index_unlinked <- new$index

columns <- c(
  "state_code", "sector", "period", "index", "index_unlinked", "price_source",
  "link_factor", "link_months", "link_ratio_mad", "link_ratio_min", "link_ratio_max"
)
index <- rbind(old[columns], new[columns])
index$year <- as.integer(format(index$period, "%Y"))
index$month <- as.integer(format(index$period, "%m"))
index <- index[order(index$state_code, index$sector, index$period), , drop = FALSE]
rownames(index) <- NULL
validate_temporal_price_chain(index, switch_date)

structure(
  list(index = index, links = links, switch_date = switch_date),
  class = "emi_temporal_price_series"
)
}

```

Cross-vintage district candidate scoring

File: R/districts/lineage_matching.R

```

score_match_candidates <- function(source_roster, reference_units, excluded_source_ids = character()) {
  need_pkg("stringdist", "district-lineage candidate scores")
  source_roster <- safe_df(source_roster)
  reference_units <- safe_df(reference_units)
  if (!"raw_district" %in% names(source_roster)) source_roster$raw_district <- source_roster$district
  source_open <- source_roster[!source_roster$source_row_id %in% excluded_source_ids, , drop = FALSE]
  if (!nrow(source_open)) return(empty_source_candidates())

  pairs <- merge(
    source_open, reference_units,
    by = "state_std", all = FALSE, sort = FALSE,

```

```

    suffixes = c("_source", "_candidate")
  )
  if (!nrow(pairs)) return(empty_source_candidates())
  source_name <- pairs$district_std_source
  candidate_name <- pairs$district_std_candidate
  pairs$jw <- 1 - stringdist::stringdist(source_name, candidate_name, method = "jw", p = 0.1)
  dl_distance <- stringdist::stringdist(source_name, candidate_name, method = "dl")
  denom <- pmax(nchar(source_name), nchar(candidate_name), 1L)
  pairs$dl <- pmax(0, 1 - dl_distance / denom)
  pairs$trigram <- stringdist::stringsim(source_name, candidate_name, method = "cosine", q = 3)
  pairs$token <- token_jaccard_similarity(source_name, candidate_name)
  pairs$score <- rowMeans(pairs[c("jw", "dl", "trigram", "token")], na.rm = TRUE)

  ranked <- safe_bind_rows(lapply(split(seq_len(nrow(pairs)), pairs$source_row_id), function(i) {
    x <- pairs[i, , drop = FALSE]
    preference <- vintage_preference(x$wave[[1]])
    vintage_rank <- match(x$reference_vintage, preference, nomatch = length(preference) + 1L)
    x <- x[order(-x$score, -x$jw, vintage_rank, x$unit_id), , drop = FALSE]
    x$rank <- match(x$district_std_candidate, unique(x$district_std_candidate))
    distinct_scores <- x$score[!duplicated(x$district_std_candidate)]
    second <- if (length(distinct_scores) >= 2L) distinct_scores[[2]] else -Inf
    x$margin <- ifelse(x$rank == 1L, x$score - second, NA_real_)
    x$preferred_vintage <- !duplicated(x$district_std_candidate)
    x
  })))

  candidate_groups <- split(seq_len(nrow(ranked)), ranked$unit_id)
  best_by_candidate <- stats::setNames(
    vapply(candidate_groups, function(i) {
      score <- ranked$score[i]
      ranked$source_row_id[i[which.max(score)]]
    }, character(1)),
    names(candidate_groups)
  )
  ranked$reciprocal_nearest <- ranked$rank == 1L &
    unname(best_by_candidate[ranked$unit_id]) == ranked$source_row_id
  threshold <- district_match_candidate_thresholds()
  ranked$high_precision_candidate <- ranked$rank == 1L & ranked$preferred_vintage &
    ranked$reciprocal_nearest &
    directional_tokens_compatible(ranked$district_std_source, ranked$district_std_candidate) &
    ranked$jw >= threshold[["jw"]] & ranked$dl >= threshold[["dl"]] &
    ranked$trigram >= threshold[["trigram"]] & ranked$margin >= threshold[["margin"]]
  ranked <- ranked[ranked$rank <= 5L, , drop = FALSE]

  data.frame(
    source_row_id = ranked$source_row_id,
    wave = ranked$wave,
    source_code = ranked$source_code,
    state_std = ranked$state_std,
    source_name_raw = ranked$raw_district,

```

```

source_name = ranked$district_std_source,
candidate_unit = ranked$unit_id,
candidate_name = ranked$district_std_candidate,
candidate_source_id = ranked$source_id,
reference_vintage = ranked$reference_vintage,
candidate_method = "fuzzy_name_candidate",
rank = ranked$rank,
jw = ranked$jw,
dl = ranked$dl,
trigram = ranked$trigram,
token = ranked$token,
score = ranked$score,
margin = ranked$margin,
reciprocal_nearest = ranked$reciprocal_nearest,
high_precision_candidate = ranked$high_precision_candidate,
stringsAsFactors = FALSE
)
}

```